

Lezione 1: Compiler, Linker e Preprocessamento

Obiettivi della lezione

Al termine di questa lezione, lo studente sarà in grado di:

- Comprendere il funzionamento del **preprocessor**, del **compiler** e del **linker**.
 - Conoscere le fasi di traduzione del codice sorgente in un programma eseguibile.
 - Analizzare il ruolo specifico di ciascun componente all'interno della toolchain di sviluppo.
-

Introduzione

Quando scriviamo un programma in un linguaggio di alto livello come C o C++, il codice sorgente non viene eseguito direttamente dal processore. Il percorso che porta un file sorgente (es. `.c` o `.cpp`) a diventare un eseguibile richiede diversi passaggi, che includono **preprocessing**, **compilazione**, **assemblaggio** e **linking**. Questa lezione si concentra su tre elementi fondamentali: **preprocessor**, **compiler** e **linker**.

1. Il Preprocessor

Il **preprocessor** ("pre-processore") è la prima fase del processo di compilazione. Lavora sul file sorgente prima che venga compilato, eseguendo direttive specifiche che iniziano con il simbolo `#`.

Compiti principali del Preprocessor

1. Inclusione dei file di intestazione:

- Sostituisce direttive come `#include <header.h>` con il contenuto reale dei file di intestazione.
- Esempio:

```
#include <stdio.h>
```

Viene sostituito con il codice contenuto in `stdio.h`.

2. Definizione delle macro:

- Sostituisce le macro definite tramite `#define`.
- Esempio:

```
#define PI 3.14
```

Ogni occorrenza di `PI` nel codice sorgente viene rimpiazzata da `3.14`.

3. **Condizionali di compilazione:**

- Include o esclude parti di codice in base a condizioni definite dall'utente.

Esempio:

```
#ifdef DEBUG
    printf("Debug mode attivo\n");
#endif
```

4. **Eliminazione di commenti:**

- Rimuove i commenti dal codice sorgente.

Output del Preprocessor

- Genera un file intermedio (spesso con estensione `.i` o `.ii`) che contiene codice sorgente senza direttive del preprocessore.
-

2. Il Compiler

Il **compiler** ("compilatore") traduce il codice sorgente preprocessato in un linguaggio di basso livello, come il linguaggio assembly o codice macchina intermedio (IR).

Fasi del Compiler

1. **Analisi lessicale:**
 - Divide il codice sorgente in unità sintattiche chiamate *token* (parole chiave, identificatori, operatori, ecc.).
2. **Analisi sintattica:**
 - Verifica che la struttura del codice sorgente rispetti le regole grammaticali del linguaggio.
 - Crea una rappresentazione interna chiamata *albero sintattico*.
3. **Analisi semantica:**
 - Controlla il significato del codice (es. tipi di variabili corretti, numero di parametri in una funzione).
4. **Ottimizzazione:**
 - Effettua miglioramenti al codice intermedio per ridurre il consumo di risorse (es. eliminazione di codice ridondante).
5. **Generazione del codice:**
 - Traduce il codice intermedio in linguaggio assembly o codice macchina specifico per l'architettura target.

Output del Compiler

- Produce un file oggetto (es. `main.o`), che contiene codice macchina binario non ancora eseguibile.
-

3. Il Linker

Il **linker** ha il compito di combinare uno o più file oggetto in un unico programma eseguibile.

Tipi di Linking

1. **Linking statico:**
 - Il codice delle librerie richieste viene copiato direttamente nell'eseguibile finale.
 - Pro: L'eseguibile è indipendente da librerie esterne.
 - Contro: L'eseguibile può essere più grande.
2. **Linking dinamico:**
 - L'eseguibile fa riferimento a librerie esterne (es. `.so` su Linux o `.dll` su Windows).
 - Pro: Riduce la dimensione dell'eseguibile.
 - Contro: Richiede che le librerie siano disponibili sul sistema in cui il programma viene eseguito.

Compiti del Linker

1. **Risoluzione dei simboli:**
 - Associa i riferimenti alle funzioni e alle variabili ai loro indirizzi reali.
 - Esempio: La chiamata a `printf()` viene collegata all'indirizzo della funzione nella libreria standard.
2. **Risoluzione delle dipendenze:**
 - Gestisce le dipendenze tra moduli, assicurandosi che tutte le funzioni e variabili siano definite correttamente.
3. **Creazione dell'eseguibile:**
 - Combina i file oggetto e le librerie in un file binario pronto per essere eseguito.

Output del Linker

- Un file eseguibile (es. `program.exe` o `program.out`).
-

Esempio Pratico

Prendiamo il seguente codice:

```
#include <stdio.h>
#define PI 3.14

int main() {
    float raggio = 5.0;
    float area = PI * raggio * raggio;
    printf("Area del cerchio: %.2f\n", area);
    return 0;
}
```

Fasi di Traduzione

1. Preprocessing:

- Il preprocessore sostituisce `#include <stdio.h>` con il contenuto del file `stdio.h` e `PI` con `3.14`.

2. Compilazione:

- Il compilatore traduce il codice in un file oggetto (es. `main.o`).

3. Linking:

- Il linker collega `main.o` con le librerie standard (es. `libc`) per produrre l'eseguibile finale.

Esempio completo - dal codice sorgente al codice eseguibile

Ecco un esempio completo che illustra il flusso di preprocessing, compilazione, assemblaggio e linking per un programma in C semplice, utilizzando una libreria statica.

Abbiamo un programma principale (`main.c`) e una libreria statica che contiene una funzione.

File: [main.c](#)

```
#include <stdio.h>

#include "math_utils.h" // Header della libreria statica

int main() {

    int a = 5, b = 3;

    int result = add(a, b); // Chiama la funzione dalla libreria

    printf("La somma di %d e %d è %d\n", a, b, result);

    return 0;

}
```

File: [math_utils.h](#)

```
#ifndef MATH_UTILS_H

    #define MATH_UTILS_H

    int add(int a, int b);

#endif
```

File: [math_utils.c](#)

```
#include "math_utils.h"

int add(int a, int b) {

    return a + b;

}
```

Riepilogo dei Passaggi e Output

Fase	Input	Output	Comando
Preprocessamento	main.c, math_utils.c, math_utils.h	main.i, math_utils.i	gcc -E
Compilazione	main.i, math_utils.i	main.s, math_utils.s	gcc -S
Assemblaggio	main.s, math_utils.s	main.o, math_utils.o	gcc -c
Libreria Statica	math_utils.o	libmathutils.a	ar rcs
Linking	main.o, libmathutils.a	program	gcc

Questo esempio mostra come ogni passaggio contribuisce a trasformare il codice sorgente in un eseguibile, chiarendo il ruolo di preprocessore, compilatore, assembler e linker.

Conclusione

Il **preprocessor**, il **compiler** e il **linker** sono strumenti fondamentali nella toolchain di sviluppo. Ciascuno svolge un ruolo specifico e contribuisce alla trasformazione del codice sorgente in un programma eseguibile. Comprendere il funzionamento di queste componenti è essenziale per scrivere codice efficiente e per risolvere problemi legati alla compilazione o al linking.

Domande per gli Studenti

1. Qual è la differenza tra linking statico e linking dinamico?
2. Perché è importante la fase di preprocessing?
3. Quali errori possono verificarsi durante il linking?

Compiti per Casa

1. Scrivere un semplice programma C che utilizza una libreria esterna e osservare le fasi di compilazione e linking.
2. Esaminare un file preprocessato (usando il corrispettivo comando Windows di gcc -E su Linux) per comprendere l'output del preprocessore.